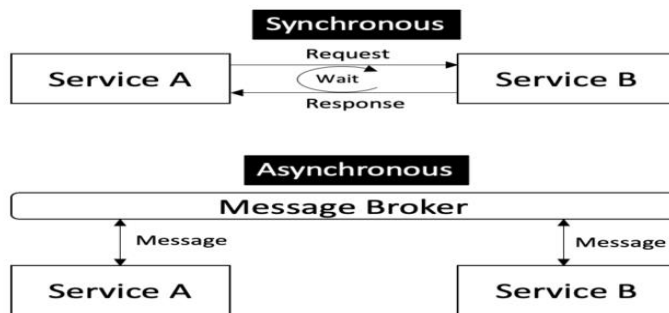


Microservice Communication

Communication Types



1. Synchronous Communication: (Request/Response):

- ✓ **Protocol:** HTTP/REST, gRPC.
- ✓ **Characteristics:** Real-time, **blocking calls**.
- ✓ **Example:** Service A calls Service B's API and waits for a response.

Pros: Simpler to implement. Useful for critical operations or user interactions requiring immediate feedback.

Cons: Can block Service A if Service B is slow.

2. Asynchronous Communication: (Message Passing):

- ✓ **Protocol:** Messaging queues (**RabbitMQ, Kafka, AWS SQS**).
- ✓ **Characteristics:** Non-blocking, event-driven.
- ✓ **Example:** Service A sends a message to a queue/topic, and Service B processes it later.

Pros: Decouples services, making them more independent. So better for long-running or background tasks.

Cons: Requires additional infrastructure (message broker).

Choosing the Right Approach

- **Use synchronous communication for:** Critical operations, user interactions needing immediate feedback, simple workflows.
- **Use asynchronous communication for:** Background tasks, data processing, notifications, decoupling services for scalability.

Many microservices architectures utilize a combination of both approaches for optimal performance and flexibility.

Comparison

Feature	RestTemplate	WebClient	OpenFeign
Use Case	Legacy systems	Reactive systems	Microservices
Blocking	Yes	No	N/A (Declarative)
Reactive	No	Yes	Yes (with WebClient under the hood)
Best For	Simple apps	Microservices, Reactive apps	Microservice communication

When to Use What:

- **RestTemplate:** For simple, blocking HTTP calls in non-reactive applications (legacy code, small projects). Generally avoid for new development.
- **WebClient:** For modern, high-performance, reactive applications and microservices.
- **OpenFeign:** For simplifying communication between microservices by abstracting away HTTP details. Recommended for most microservice-to-microservice communication.

Steps for Using RestTemplate, WebClient, and OpenFeign

RestTemplate: it is Synchronous, blocking.

1. Add Spring Web Dependency

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

2. Configure RestTemplate as a Spring Bean

```
@Bean
RestTemplate restTemplate() {
    return new RestTemplate();
}
```

3. Inject and Use RestTemplate to Call REST API

```
@Autowired
RestTemplate restTemplate;
```

```
String result = restTemplate.getForObject("http://localhost:8082/ms2", String.class);
return "The data from ms2: " + result;
```

Note: RestTemplate is Marked as **deprecated** for reactive use cases in Spring Boot 2.4+. Replace with WebClient for new projects. Some common methods
getForObject() / getForEntity() - For GET requests
exchange() - For custom requests with HTTP methods.

2. **WebClient:** Modern, non-blocking HTTP client in Spring WebFlux.
Designed for reactive applications. Ideal for microservices.

1. **Add Spring WebFlux Dependency**

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

2. **Configure WebClient as a Spring Bean**

```
@Bean
WebClient webClient() {
    return WebClient.builder().build();
}
```

3. **Inject and Use WebClient to Call REST API**

```
@Autowired
WebClient webClient;

String result = webClient.get()
    .uri("http://localhost:8082/ms2")
    .retrieve()
    .bodyToMono(String.class)
    .block(); // Use block() for synchronous response
return "The data from ms2: " + result;
```

3. OpenFeign: Declarative, easier for microservices. Abstracts HTTP calls into Java interfaces. Use with Spring Cloud for service discovery and load balancing.

1. Add Spring Cloud OpenFeign Dependency

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

2. Enable Feign Clients

Use **@EnableFeignClients** in your Spring Boot main application class:

```
@SpringBootApplication
@EnableFeignClients
public class MyApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyApplication.class, args);
    }
}
```

3. Create FeignClient Interface (inside config package)

```
@FeignClient(name = "myfeign", url = "http://localhost:8082")
public interface MyFeignClient {
    @GetMapping("ms2")
    String meth1();
}
```

4. Inject and Use FeignClient to Call REST API

```
@Autowired
MyFeignClient myFeignClient;

String result = myFeignClient.meth1();
return "The data from ms2: " + result;
```